Defensive Disclosure: A deterministic evidence protocol that binds every approval to an immutable source digest.

1. Identification

Record ID: `ARTIFEX-20261017-9124A48A25C8`
Origin ID: `ARTIFEX-20261017-8DB4B454A577`
Origin recorded UTC: `2026-10-17T20:52:49Z`
Origin hash: `8db4b454a57788324bb175d7a28fdfadc836b69fc4fd25d799c911dfe9558040`
Candidate score: `16`
Evidence scope: `historical_inventory`

This disclosure is based on a historical inventory. Artifex does not assert that human origin was recorded before AI assistance or before the artifact existed.

2. Human Conception Record

A deterministic evidence protocol that binds every approval to an immutable source digest.

The complete signed-off conception record is included as `conception_record.md`.

3. Technical Problem

A deterministic evidence protocol that binds every approval to an immutable source digest.

4. Technical Solution

The captured candidate at protocol.py supplies the technical mechanism:

```
class EvidenceProtocol:
    def verify(self, artifact, digest):
        return artifact, digest
```

Implementation explanation: this architecture uses a deterministic verification workflow because each transition must preserve the source digest and immutable invariant. The technical design supplies a state machine, configuration schema, and enablement notes so that another engineer can implement the mechanism.

5. Implementation Details

Captured source: `protocol.py`
Captured source SHA-256: `b17db39b516851cc8a140f479468c49522a2025826f07a1d4d3a7cde3f6cb2a7`

The following technical artifact was captured, hashed, and bound to the human origin record. It is included so the operative mechanism can be evaluated without relying

on an unsupported summary.

> class EvidenceProtocol:
>   def verify(self, artifact, digest):
>   return artifact, digest
>
> Implementation explanation: this architecture uses a deterministic verification
> workflow because each transition must preserve the source digest and immutable
> invariant. The technical design supplies a state machine, configuration schema,
> and enablement notes so that another engineer can implement the mechanism.

6. Essential Components

The captured candidate at protocol.py and its source manifest identify these implementation details:

```
class EvidenceProtocol:
    def verify(self, artifact, digest):
        return artifact, digest
```

Implementation explanation: this architecture uses a deterministic verification workflow because each transition must preserve the source digest and immutable invariant. The technical design supplies a state machine, configuration schema, and enablement notes so that another engineer can implement the mechanism.

7. AI Assistance Summary

Artifex captured this artifact during a historical inventory; the record does not infer when AI assistance occurred or which specific parts AI produced.

Human decisions, designs, or changes: The user described the work as: A deterministic evidence protocol that binds every approval to an immutable source digest. Artifex makes no inventorship determination.

8. Source Hashes

See `source_hashes.json`. Algorithm: SHA-256.

9. Publication Hash

`a480c4b819d3667c46a1b337c9e5c6de5549dcf591028c26746612cead7efe6e`

10. Public Ledger Location

https://github.com/electricwolfemarshmallowhypertext/artifex-publication-validation/tree/artifex/ARTIFEX-20261017-9124A48A25C8/disclosures/ARTIFEX-20261017-9124A48A25C8

11. Defensive Disclosure Notice

This document is intentionally published as defensive prior art. It describes the technical subject matter so that the public can identify and implement the disclosed invention. It is not legal advice and makes no representation about patentability or ownership.